

CPT304 Software Engineering 2 - Tutorial 3

Practical

1. Extend the Burger Restaurant. Base on the full factory example in the lecture notes, extend the code following Factory Pattern to accept the ordering of Chicken Burger.
2. A document management system in which documents can be in different formats like Word, PDF, Excel, etc. The documents need to be created, opened, edited and saved. How do we organize our object-oriented system to handle this diversity, keeping in mind that in the future we may have to introduce more document types? Write a factory design pattern to encapsulate the objects initialization process.
3. Decorator design pattern: A new type of Robot is invented. It is smarter because it can apply deep learning to cook better. Now, any Robot can be a rational robot at the same time smart robot. Enhance the example in the lecture notes by implement a SmartRobotDecorator. Illustrate that a Robot can be rational and smart at the same time.
4. Let's think about a cafe scenario. When you go to a cafe, you usually start with a basic type of coffee, then you customize it by adding extra features like milk, sugar, flavors, whipped cream, etc. Plus, each of these additions affects the final cost of the coffee. Implementing this in a straightforward way could quickly become complicated and messy, especially if you want to be able to add, remove, and modify these customizations dynamically. This is a perfect scenario for the Decorator pattern. In this pattern, a class represents the base object (in this case, a plain coffee), while other decorator classes represent the optional extras. The decorators wrap the base class and other decorators, forming a chain of objects where each object adds its own behavior (and cost) to the final product.
5. Consider an e-commerce scenario where users can make payments using different payment systems like PayPal, CreditCards, and cryptocurrencies. Each of these payment systems has a different API and integration process. When a user chooses a payment method, your system should process the payment using the respective payment system's API. Directly integrating these different APIs into your system can lead to a lot of complexities and code clutter. This is a perfect scenario to use the Adapter Pattern. Show and discuss your implementation.

Theory

Question 1

Let's assume you have an application where you need to write logs to a file. This could be anything from info logs, debug logs to error logs.

The following needs are to be addressed here:

1. All parts of an application need to write to the same log file.
2. Only one part of the application should be able to write to the log file at a time to avoid concurrency issues.

Which design pattern should you use as the solution for the problem?

Question 2

Imagine a scenario where you are developing a video streaming service, similar to Netflix or YouTube.

Streaming a video is a resource-intensive operation, especially for high-definition videos. If your service were to directly stream videos from their original location for every single user request, it could lead to a lot of latency, high bandwidth usage, and potentially overloading the server.

Instead of directly streaming the video from its original location, you initially only holds the reference to the original video. When the user clicks on a video to watch, you will then streams the video from the original location, buffers it, and sends the buffered data to the client. Which design pattern is suitable for the problem?

Question 3

Consider a scenario where you're developing an application for a Vehicle Rental Service. This service can rent out several types of vehicles like cars, trucks, and motorcycles.

Customers can select the type of vehicle they want to rent. Based on the customer's choice, the system must create an object of that type. If you were to create these objects directly within your application, it could lead to a lot of repetitive, error-prone code, and it would make your code harder to maintain and extend. Which design pattern is most suited to this scenario?

Question 4

Consider a scenario in which you're designing a data analytics tool that uses data from multiple sources. These sources could include different databases (SQL, NoSQL), CSV files, and even APIs that return JSON.

Each of these data sources has a different way of delivering the data. SQL databases use SQL queries, NoSQL databases have their own query syntax, CSV files need a file reader and JSON APIs require HTTP requests. Moreover, the returned data from each source is in a different format.

If you were to integrate every data source directly into your system, it would result in a lot of redundant and hard-to-maintain code, as you would have to handle each data source and format separately. Which design pattern is suitable for the problem?

Question 5

Consider a file system. In a file system, there are two types of entities: `File` and `Directory`.

A `File` is a leaf node that has content and size, while a `Directory` can contain multiple files or directories but doesn't have content of its own, its size is typically the total of sizes of all files it contains.

To develop the file system described above, which design pattern is most suitable for the problem?

Question 6

Consider a scenario where a text-editing application that allows users to apply different types of formatting to the text.

Let's say that you have plain text and you want to allow users to apply bold, italic, underline, or strikethrough formatting. The user should be able to apply these formats independently or in combination. Which design pattern is the most suited solution for the problem?